HW4: Problem-3 - SSL Certificates and X.509

Overview: The Trust Framework

In the digital world, we need a way to prove that a website is who it claims to be. This is achieved through the Public Key Infrastructure (PKI) and the X.509 standard, which act as a global "trust system" for the internet.

1 Understanding the X.509 Standard

X.509 is a digital certificate that is built on top of a widely trusted standard known as ITU or International Telecommunication Union X.509 standard, in which the format of PKI certificates is defined. X.509 digital certificate is a certificate-based authentication security framework that can be used for providing secure transaction processing and private information. These are primarily used for handling the security and identity in computer networking and internet-based communications.

Think of an X.509 certificate as a digital identity card for a website. Just as a government issues a physical ID to a citizen, a trusted Certificate Authority (CA) issues an X.509 certificate to a web server.

Key Components of the Certificate

Every X.509 certificate contains essential "ID details," including:

- **Version & Serial Number:** Unique identifiers for the specific document.
- **Issuer Name:** The trusted authority (like Google Trust Services) that verified the site.
- **Validity Period:** The "Start" and "Expiry" dates of the certificate.
- **Subject Name:** The identity being verified (e.g., `*.google.com`).
- **Signature:** A hash of all the data, encrypted by the CA, to prove the certificate hasn't been tampered with.

Common Storage Formats

These "digital IDs" are stored in different file formats depending on the system:

- **.PEM:** The most common text-based format (readable in Notepad).
- **.DER:** A compact binary version used by specific systems like Java.

- **.P12/.PFX:** A secure, password-protected file that holds the ID and the secret "key" together.

2 Practical: Fingerprint Generation via OpenSSL

Using the Linux terminal (WSL), I retrieved the live certificate from Google's server and generated its unique Fingerprint.

Listing 1: Generating Certificate Fingerprint

```
# Step 1: Download the certificate from google.com
echo | openssl s_client -connect google.com:443 | \
sed -ne '/-BEGIN␣CERTIFICATE-/,/-END␣CERTIFICATE-/p' >
   google_cert.pem

# Step 2: Generate the SHA1 fingerprint
openssl x509 -noout -fingerprint -sha1 -inform pem -in
   google_cert.pem
```

Generated SHA1 Fingerprint:

```
sha1 Fingerprint=F8:90:A0:F9:74:3F:20:44:F3:3A:FB:A7:89:DC:37:89:3C:83:4D:03
```

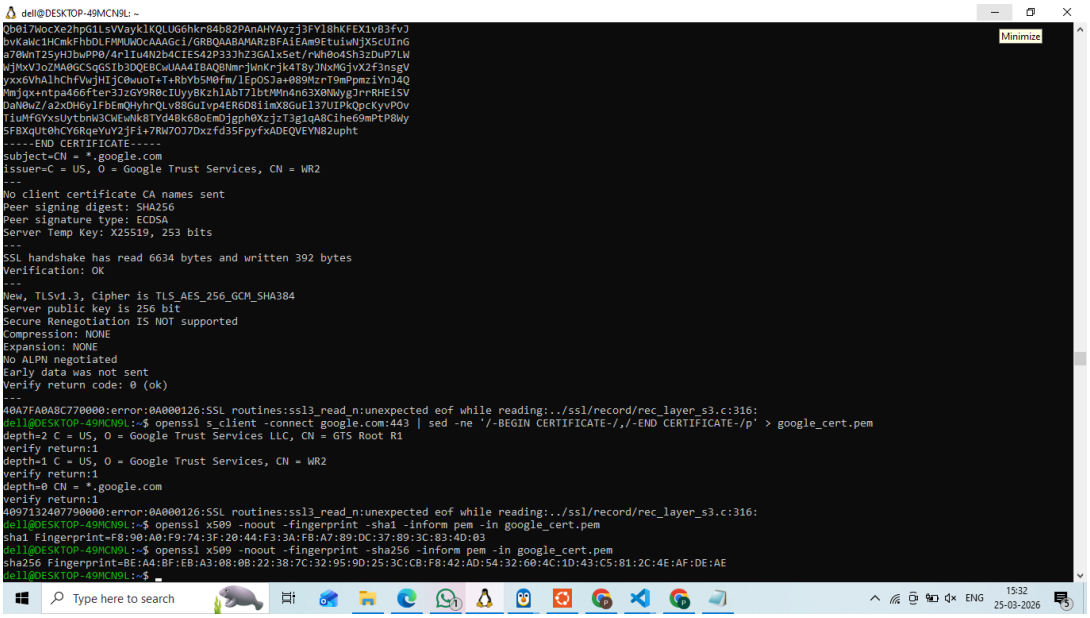

Figure 1: WSL Terminal: OpenSSL certificate download and fingerprint generation.

3 Browser Verification

I cross-checked this value by opening the certificate in my web browser. The "Thumbprint" matched the fingerprint generated in my terminal, confirming that the certificate I downloaded is the same authentic one being used by the browser to secure the connection.

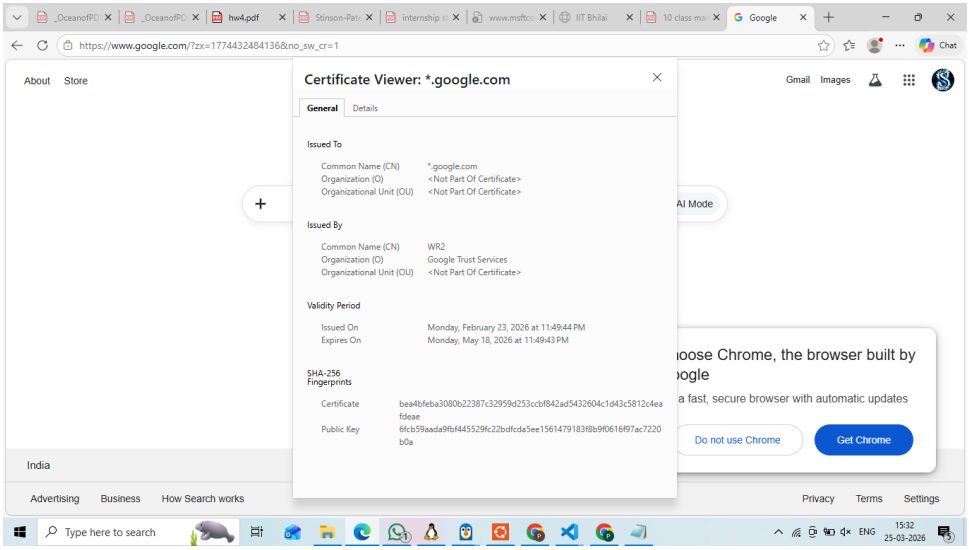

Figure 2: Browser Certificate Viewer highlighting the SHA-256 and Thumbprint fields.